

Computer Data Security and Privacy
Academic Year (2025-2026) - Second Semester
CYS 313 / CIS 425 -Year 3
Project Supervisor: Dr. Ghada Bilal
Group Number (1)
(Project Report)

#	Student Name	Student ID
1	Shahad Mohammed Aldossari	2240004665
2	Batool Alabdullah	2240003056
3	Zahra Ali Alabkari	2240001738
4	Sarah Aljabran	2240007036
5	Fatimah Almassari	2240004117
6	Rania Sanad	2240001545

Table of Contents

1. Introduction	3
2. Tools Analysis	3
2.1 Zenmap (Information Gathering)	3
2.1.1 Tool Presentation & Typical Use-Case	3
2.1.2 Used Procedure	3
2.1.3 Result and Analysis	5
2.1.4 Countermeasures	5
2.2 Cisco Global Exploiter (Vulnerability Analysis)	6
2.2.1 Tool Presentation & Typical Use-Case	6
2.2.2 Used Procedure	7
2.2.3 Result and Analysis	8
2.2.4 Countermeasures	9
2.3 Johnny (Password Cracking)	9
2.3.1 Tool Presentation & Typical Use-Case	9
2.3.2 Used Procedure	10
2.3.3 Result and Analysis	10
2.3.4 Countermeasures	11
2.4 OWASP ZAP (Website Penetration Testing)	12
2.4.1 Tool Presentation & Typical Use-Case	12
2.4.2 Used Procedure	12
2.4.3 Result and Analysis	13
2.4.4 Countermeasures.....	14
3. Conclusion	15

Figures Table

Figure	3
Figure 2	4
Figure 3	4
Figure 4	5
Figure 5	6
Figure 6	7
Figure 7	7
Figure 8	7
Figure 9	8
Figure 10	8
Figure 11	11
Figure 12	12
Figure 13	13
Figure 14	13
Figure 15	14
Figure 16	14

1. Introduction

In this project, we study Kali Linux and its powerful tools for security testing. Our goal is to learn how to find security weaknesses and how to fix them. We picked four tools from different areas: **Zenmap** to scan networks, **Cisco Global Exploiter** to test system vulnerabilities, **Johnny** to check password strength, and **OWASP ZAP** to secure websites. By using these tools, we show how a security audit works and provide tips to keep systems safe.

2. Tools

2.1 Zenmap

2.1.1 Tool presentation

Zenmap is a graphical user interface for Nmap. It is used to scan networks, detect open ports, and identify running services. It helps security analysts discover potential vulnerabilities and analyze network security risks. Zenmap is commonly used by cybersecurity professionals and penetration testers to analyze network security and identify possible vulnerabilities.

2.1.2 Used procedure

1. Opened Zenmap in Kali Linux.
 2. Entered the target IP address (127.0.0.1).
 3. Selected the scan profile: Intense Scan.
- Command used: `nmap -T4 -A -v 127.0.0.1`

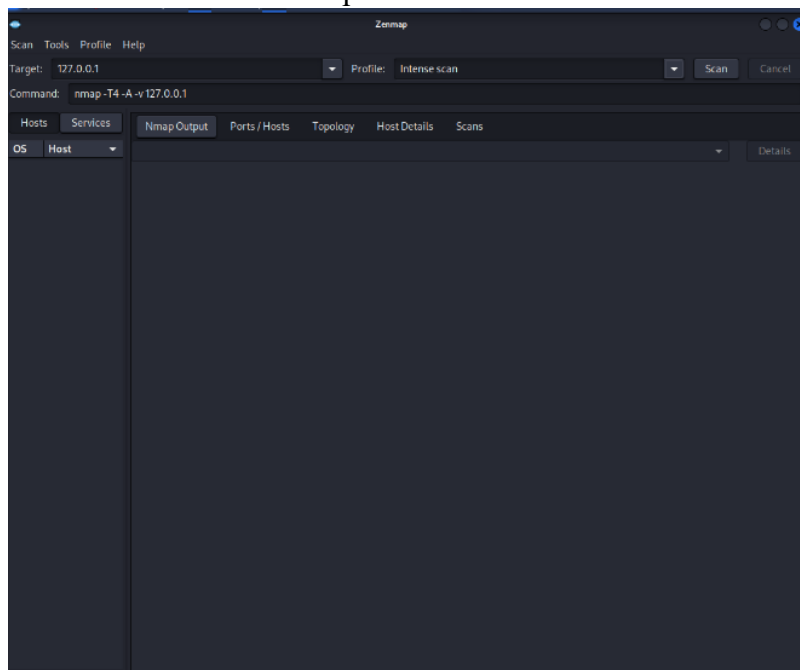


Figure 1: Zenmap target and Intense scan configuration.

Computer Data Security and Privacy Project: Kali Linux Tools Analysis

- Clicked on “Scan” to start the process.
- Waited for the scan to complete.

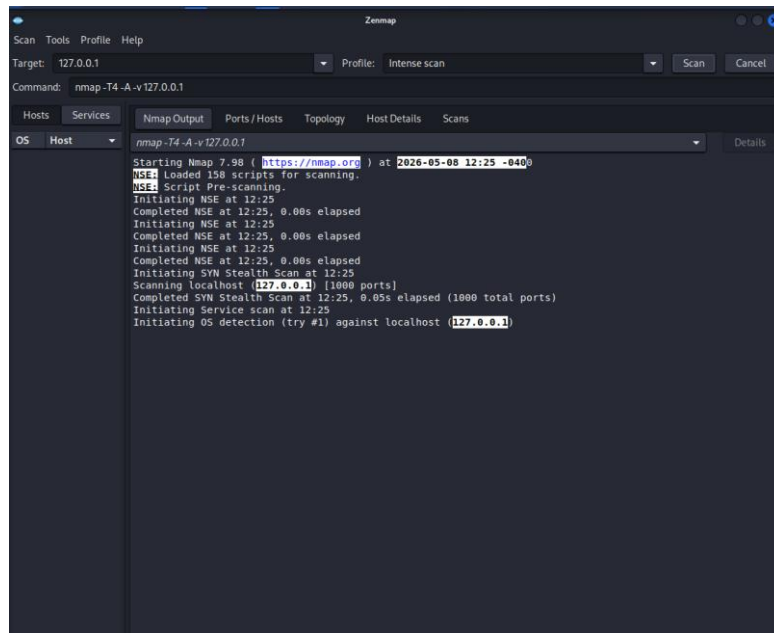


Figure 2: Zenmap scanning process.

- Observed the scan results including ports and services.

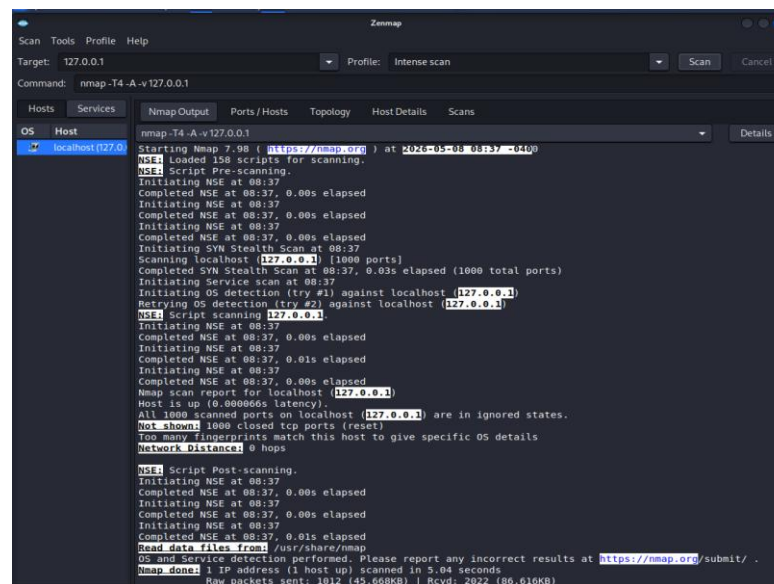
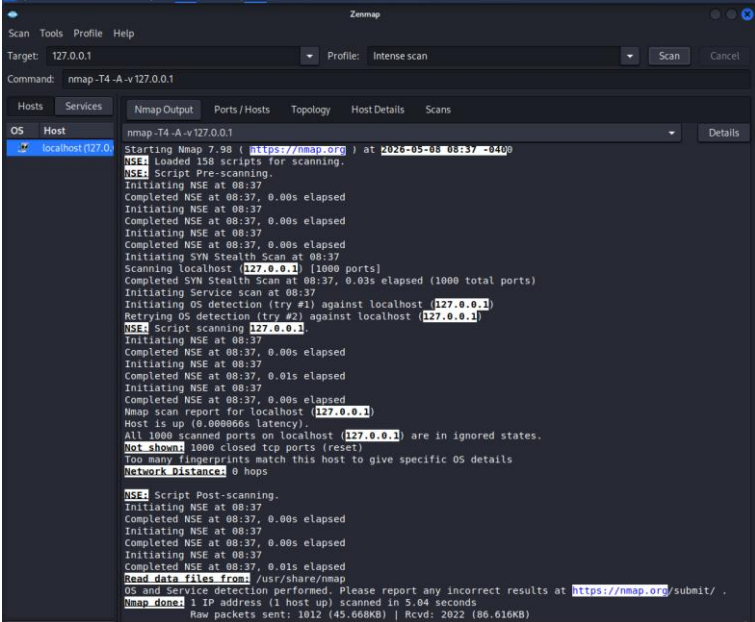


Figure 3: Scan results showing closed ports.

2.1.3 Result and Analysis



```
Starting Nmap 7.90 ( https://nmap.org ) at 2020-05-08 08:37 -0400
NSE: Loaded 158 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 08:37
Completed NSE at 08:37, 0.00s elapsed
Initiating NSE at 08:37
Completed NSE at 08:37, 0.00s elapsed
Initiating SYN Stealth Scan at 08:37
Scanning localhost (127.0.0.1) [1000 ports]
Completed SYN Stealth Scan at 08:37, 0.03s elapsed (1000 total ports)
Initiating Service scan at 08:37
Initiating OS detection (try #1) against localhost (127.0.0.1)
Retrying OS detection (try #2) against localhost (127.0.0.1)
NSE: Script scanning 127.0.0.1
Initiating NSE at 08:37
Completed NSE at 08:37, 0.00s elapsed
Initiating NSE at 08:37
Completed NSE at 08:37, 0.01s elapsed
Initiating NSE at 08:37
Completed NSE at 08:37, 0.00s elapsed
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000000s latency).
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)
Too many fingerprints match this host to give specific OS details
Network Distance: 0 hops
NSE: Script Post-scanning.
Initiating NSE at 08:37
Completed NSE at 08:37, 0.00s elapsed
Initiating NSE at 08:37
Completed NSE at 08:37, 0.00s elapsed
Initiating NSE at 08:37
Completed NSE at 08:37, 0.01s elapsed
Read data files from /usr/share/nmap
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 5.04 seconds
Raw packets sent: 1012 (45.66KB) | Rcvd: 2022 (88.61KB)
```

Figure 4: Final scan results showing closed ports.

The scan results showed that all scanned ports on the target (127.0.0.1) are closed, and no active services were detected. This indicates that no services are accessible externally, limiting potential entry points for attackers. The presence of only closed ports reduces the attack surface and lowers the risk of external attacks. However, this does not guarantee complete security, as vulnerabilities may still exist at the system or application level and may not be detected through port scanning alone.

2.1.4 Countermeasures

- Ensure that unnecessary ports remain closed.
- Use firewall protection to control network access.
- Keep the system updated regularly.
- Perform regular network scans to detect any changes or newly opened ports.

2.2 Cisco Global Exploiter

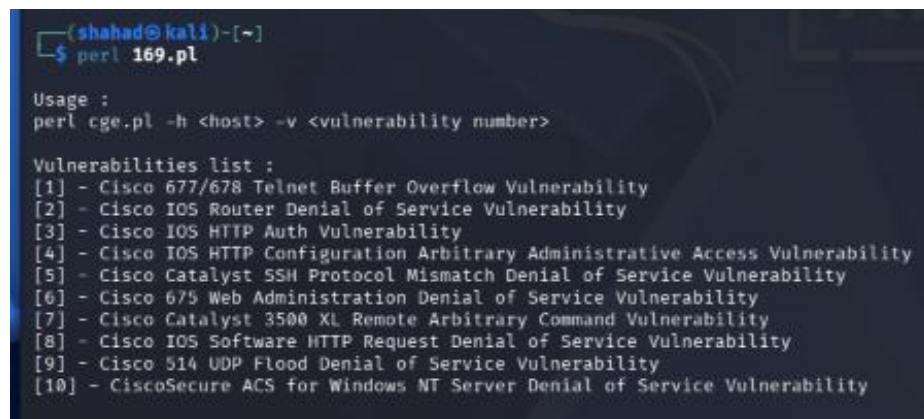
2.2.1 Tool presentation

Cisco Global Exploiter (**CGE**) is an advanced command-line tool used for remote security testing. It is specifically designed to find and exploit vulnerabilities in Cisco network devices such as routers and switches.

Main Features:

- **Vulnerability Detection:** It can identify several types of weaknesses in Cisco Systems.
- **Exploitation Modules:** It includes specific attacks like Denial of Service (**DoS**), **HTTP** server vulnerabilities, and password cracking.
- **Fast and Light:** It is a simple tool that allows security professionals to quickly check if a device is secure or needs patching.

Typical Use-Case: Security analysts use **CGE** during the "Vulnerability Analysis" or "Exploitation" phase of a penetration test to see if network infrastructure can be compromised by known exploits.



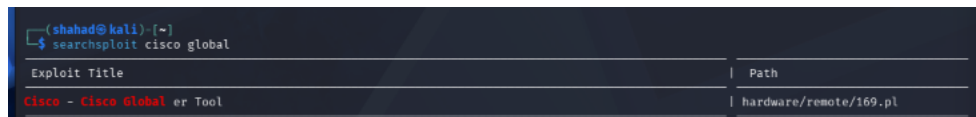
```
(shahad@kali)-[~]  
$ perl 169.pl  
  
Usage :  
perl cge.pl -h <host> -v <vulnerability number>  
  
Vulnerabilities list :  
[1] - Cisco 677/678 Telnet Buffer Overflow Vulnerability  
[2] - Cisco IOS Router Denial of Service Vulnerability  
[3] - Cisco IOS HTTP Auth Vulnerability  
[4] - Cisco IOS HTTP Configuration Arbitrary Administrative Access Vulnerability  
[5] - Cisco Catalyst SSH Protocol Mismatch Denial of Service Vulnerability  
[6] - Cisco 675 Web Administration Denial of Service Vulnerability  
[7] - Cisco Catalyst 3500 XL Remote Arbitrary Command Vulnerability  
[8] - Cisco IOS Software HTTP Request Denial of Service Vulnerability  
[9] - Cisco 514 UDP Flood Denial of Service Vulnerability  
[10] - CiscoSecure ACS for Windows NT Server Denial of Service Vulnerability
```

Figure 5: The source code of Cisco Global Exploiter (CGE) displayed in Kali Linux, listing the 10 available exploit modules for Cisco devices.

2.2.2 Used procedure

The implementation of the security testing tool followed a systematic approach within the Kali Linux environment as described below

1. **Locating the Tool:** First, the searchsploit command was used to find the Cisco Global Exploiter script within the local Exploit Database. The script 169.pl was identified as the correct Perl-based tool for this task.



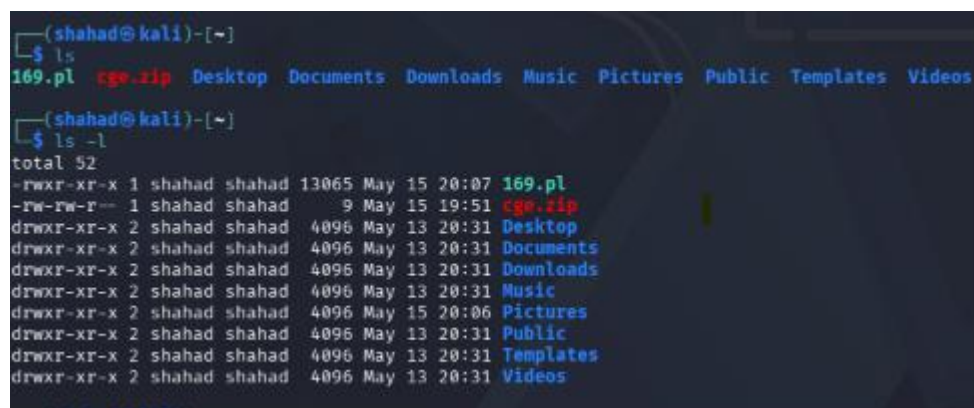
```
(shahad@kali)-[~]
$ searchsploit cisco global

```

Exploit Title	Path
Cisco - Cisco Global er Tool	hardware/remote/169.pl

Figure 6: Locating the Cisco Global Exploiter tool using the searchsploit command in Kali Linux.

2. **Preparing the Workspace:** The script was copied to the working directory. Then, the ls -l command was used to verify the file's presence and ensure that the script had the necessary execution permissions.

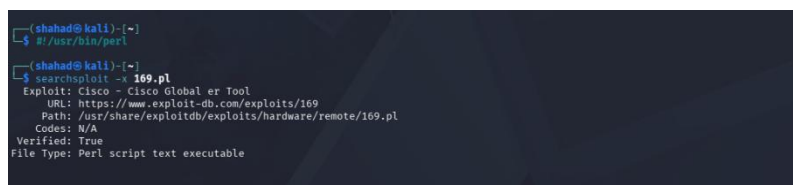


```
(shahad@kali)-[~]
$ ls
169.pl  cge.zip  Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos

(shahad@kali)-[~]
$ ls -l
total 52
-rwxr-xr-x 1 shahad shahad 13065 May 15 20:07 169.pl
-rw-rw-r-- 1 shahad shahad   9 May 15 19:51 cge.zip
drwxr-xr-x 2 shahad shahad 4096 May 13 20:31 Desktop
drwxr-xr-x 2 shahad shahad 4096 May 13 20:31 Documents
drwxr-xr-x 2 shahad shahad 4096 May 13 20:31 Downloads
drwxr-xr-x 2 shahad shahad 4096 May 13 20:31 Music
drwxr-xr-x 2 shahad shahad 4096 May 15 20:06 Pictures
drwxr-xr-x 2 shahad shahad 4096 May 13 20:31 Public
drwxr-xr-x 2 shahad shahad 4096 May 13 20:31 Templates
drwxr-xr-x 2 shahad shahad 4096 May 13 20:31 Videos
```

Figure 7: Verifying the presence of the 169.pl exploit script and its execution permissions in the working directory.

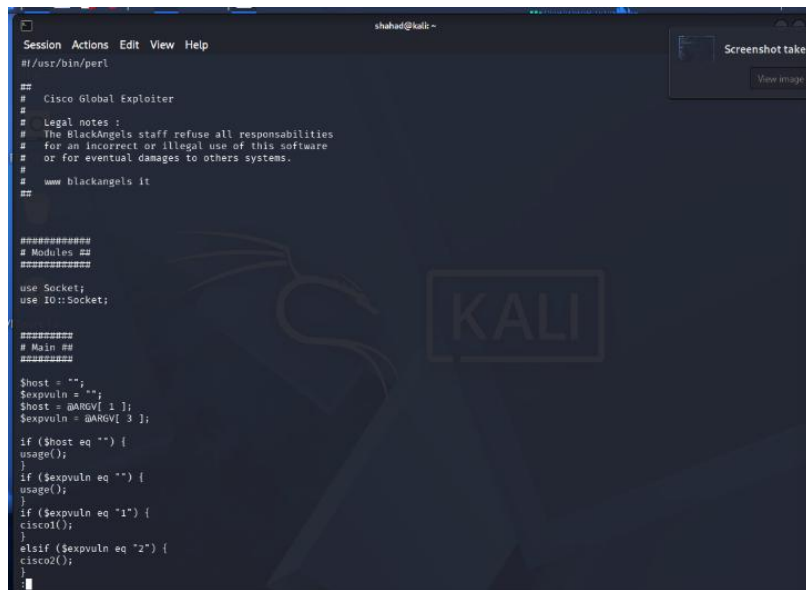
3. **Inspecting the Logic:** Before execution, the source code was briefly examined to understand how the script handles the targeting of Cisco devices and to view the available exploitation modules.



```
(shahad@kali)-[~]
$ !/usr/bin/perl

(shahad@kali)-[~]
$ searchsploit -x 169.pl
Exploit: Cisco - Cisco Global er Tool
URL: https://www.exploit-db.com/exploits/169
Path: /usr/share/exploitdb/exploits/hardware/remote/169.pl
Codes: N/A
Verified: True
File Type: Perl script text executable
```

Figure 8: Opening the Cisco Global Exploiter script (169.pl) from the local Exploit Database using the searchsploit examine option.

A screenshot of a terminal window in Kali Linux showing the source code of the Cisco Global Exploiter (CGE) tool. The terminal has a dark background with a faint Kali Linux logo. The code is displayed in a light color, showing various comments and Perl code snippets. A 'Screenshot taken' notification is visible in the top right corner.

```
shahad@kali:~$ cat /usr/bin/perl
##
## Cisco Global Exploiter
##
## Legal notes :
## The BlackAngels staff refuse all responsibilities
## for an incorrect or illegal use of this software
## or for eventual damages to others systems.
##
## www.blackangels.it
##

#####
# Modules #
#####

use Socket;
use IO::Socket;

#####
# Main #
#####

$host = "";
$expvuln = "";
$host = @ARGV[ 1 ];
$expvuln = @ARGV[ 3 ];

if ($host eq "") {
    usage();
}
if ($expvuln eq "") {
    usage();
}
if ($expvuln eq "1") {
    cisco1();
}
elsif ($expvuln eq "2") {
    cisco2();
}
}
```

Figure 9: The source code of Cisco Global Exploiter (CGE) displayed in Kali Linux for inspection and analysis.

4. **Running the Tool:** Finally, the tool was launched using the Perl interpreter with the command `perl 169.pl`. This step successfully displayed the tool's usage instructions and the list of vulnerabilities it can test.

A screenshot of a terminal window showing the execution of the Cisco Global Exploiter tool. The prompt is 'shahad@kali)~'. The command entered is 'perl 169.pl -h 192.168.1.1 -v 1'. The output is 'No telnet server detected on 192.168.1.1 ...'.

```
(shahad@kali)~)
$ perl 169.pl -h 192.168.1.1 -v 1
No telnet server detected on 192.168.1.1 ...
```

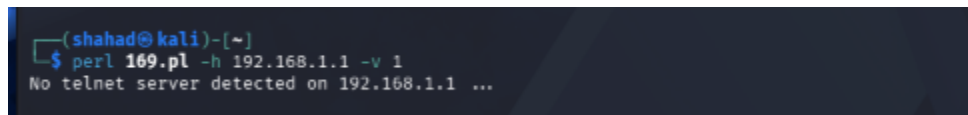
Figure 10: Running the Cisco Global Exploiter tool against a sample target IP address using vulnerability module 1 .

2.2.3 Result and Analysis

After the tool was successfully set up, a functional test was performed to evaluate the security of a target network interface (IP: 192.168.1.1). The following results were observed:

1. Execution of Exploit Module: The tool was commanded to test for a "Telnet Buffer Overflow" (Module 1) against the target host.
2. Observation: The system returned a message stating: "No telnet server detected on 192.168.1.1". This indicates that the connection attempt on the default Telnet port (23) was unsuccessful.

3. Analysis: From a security perspective, this result confirms that the target device has a reduced attack surface. Since the Telnet service is not running or is blocked by a firewall, the device is protected against any exploits targeting that specific protocol.
4. Conclusion: While the exploit did not "succeed" in compromising the device, the analysis proves the tool's effectiveness in identifying open services and verifying the security posture of Cisco infrastructure.



```
(shahad@kali)-[~]  
$ perl 169.pl -h 192.168.1.1 -v 1  
No telnet server detected on 192.168.1.1 ...
```

Figure 10: Running the Cisco Global Exploiter tool against a sample target IP address using vulnerability module 1 .

2.2.4 Countermeasures

1. Disable Insecure Services: Turn off legacy services (Telnet/HTTP) that send data in plain text.
2. Implement Secure Management: Use SSH and HTTPS to ensure all administrative traffic is encrypted.
3. Regular Patch Management: Keep the Cisco IOS updated to the latest version to protect against known vulnerabilities.
4. Access Control: Apply ACLs to restrict management access to authorized IP addresses only.

2.3 Johnny

2.3.1 Tool Presentation

Johnny is a user interface for John the Ripper. It can be used to perform password auditing by implementing different kinds of attacks such as dictionary attack and brute force attack. It simplifies the use of John the Ripper through its interface, helping ordinary users in penetration testing assess the strength of passwords and discover any weak credentials. Although the attacks will be carried out by John the Ripper alone, Johnny acts as the monitor and interface of the attacks.

2.3.2 Used procedure

1. Opened the terminal in Kali Linux.
2. Moved to the Desktop folder.
3. Created the file named “hash.txt,” which contains the hash string.
4. Examined the content of the file using the cat command.
5. Chose one wordlist (rockyou.txt).
6. Cracked the password through John the Ripper by giving the correct format.
7. Waited for the process to be completed.
8. Viewed the results of the attacked password.
9. Recorded observations and captured screenshots during the process.

2.3.3 Result and Analysis

Results:

The hashing was successfully decoded. The discovered password is: **password**

Analysis:

The selected password was weak and could be found in the frequently used dictionary list called rockyou.txt, which made it easy to crack the password.

At first, there were no results because of the incorrect format used, but once the correct format – Raw-MD5 – was set up, the hack became successful.

```
Session Actions Edit View Help
(kali@kali)-[~]
$ cd ~/Desktop
(kali@kali)-(/Desktop)
$ echo "5f4dcc3b5aa765d61d8327deb882cf99" > hash.txt
(kali@kali)-(/Desktop)
$ cat hash.txt
5f4dcc3b5aa765d61d8327deb882cf99
(kali@kali)-(/Desktop)
$ ls /usr/share/wordlists
dirb      fasttrack.txt  legion      rockyou.txt  wifite.txt
dirbuster fern-wifi      metasploit  sqlmap.txt
dnsmmap.txt john.lst      nmap.lst   wfuzz
(kali@kali)-(/Desktop)
$ sudo gzip -d /usr/share/wordlists/rockyou.txt.gz
[sudo] password for kali:
gzip: /usr/share/wordlists/rockyou.txt.gz: No such file or directory
(kali@kali)-(/Desktop)
$ john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
No password hashes left to crack (see FAQ)
(kali@kali)-(/Desktop)
$ john --format=Raw-MD5 --show hash.txt
?:password
1 password hash cracked, 0 left
```

Figure 11: Password Hash Cracking Using John the Ripper

2.3.4 Countermeasures

Have complex and strong passwords (a minimum of 12 characters).

No commonly known or dictionary based passwords should be used.

Hashing algorithms include:

- **bcrypt**

- **Argon2**

The use of salts makes password hashes secure against dictionary attacks.

2.4 ZapProxy

2.4.1 Tool presentation

assessment is **OWASP ZAP (Zed Attack Proxy)**, as indicated by the logo and interface shown in **image_6.png**. ZAP is an open-source web application security scanner intended for finding vulnerabilities in web applications during development and testing phases.

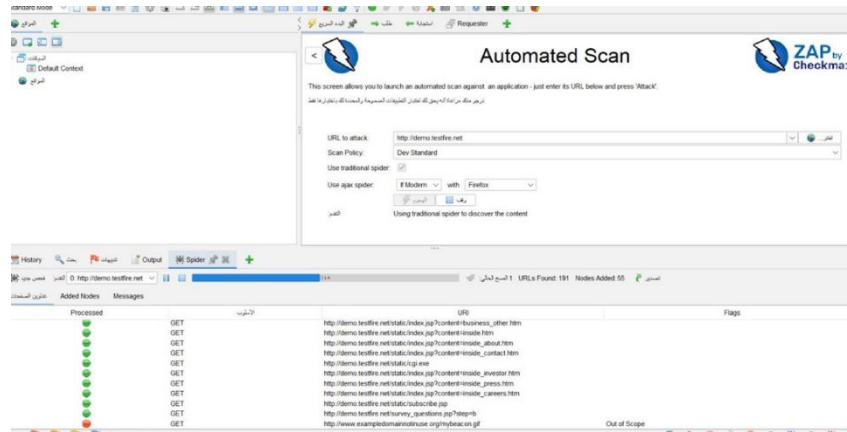


Figure 12: OWASP ZAP Automated Scan Interface and Spider Tool Execution.

2.4.2 Used procedure

1. The security assessment was performed using OWASP ZAP to identify vulnerabilities in the web application.
2. The target application was configured through the OWASP ZAP proxy.
3. Automated scans were executed to detect common vulnerabilities such as SQL Injection, XSS, missing security headers, and cookie issues.
4. Manual testing was performed to verify critical vulnerabilities.

5. HTTP requests and responses were analyzed to inspect authentication, headers, input validation, and session security.
6. All findings were documented with their severity and security impact.

2.4.3 Result and Analysis

Technical overview of the missing **Content Security Policy (CSP)** header, highlighting its role in mitigating XSS and data injection attack.



Figure 13: Content Security Policy (CSP) Header Not Set Alert

1. Evidence of a successful SQL Injection attack on the login authentication module using a logic bypass payload.
2. Evidence of the web server leaking technical version information (Apache-Coyote/1.1) within the HTTP response header fields.
3. Technical evidence of a high-risk **Reflected Cross-Site Scripting (XSS)** vulnerability, showing the input vector and the script execution payload

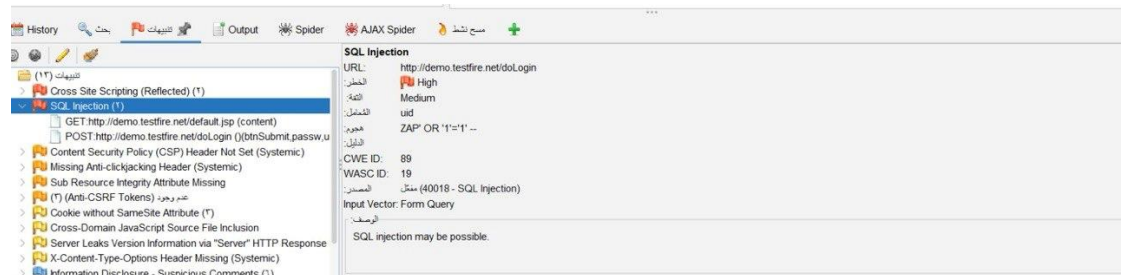


Figure 14: Vulnerability Alerts and SQL Injection Attack Details in OWASP ZAP

4. Vulnerability report showing a **Cookie without SameSite Attribute** for the JSESSIONID, which could lead to CSRF attacks.



Figure 15: Cookie without SameSite Attribute Alert

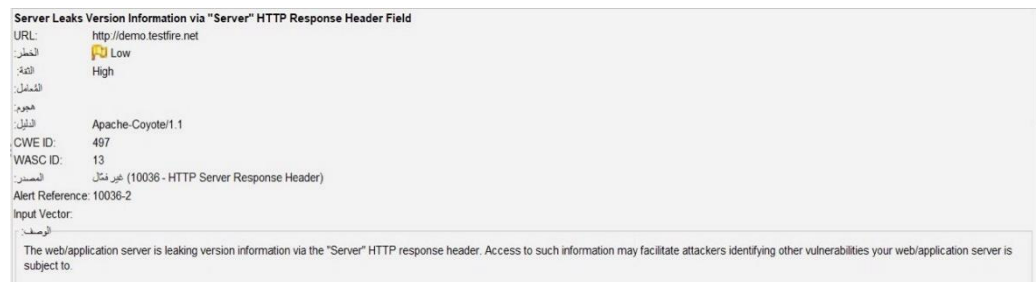


Figure 16: Server Version Information Leakage via HTTP Header

2.4.4 Countermeasures

1. Implement Content Security Policy (CSP) headers to reduce XSS attacks.
2. Use prepared statements and input validation to prevent SQL Injection.
3. Hide server version information from HTTP headers.
4. Sanitize and encode user input to prevent XSS vulnerabilities.
5. Configure cookies with Secure, HttpOnly, and SameSite attributes to improve session security and reduce CSRF risks.

3. Conclusion

In conclusion, this project provided practical experience in using Kali Linux security tools for cybersecurity testing and analysis. The selected tools helped demonstrate different aspects of security assessment, including information gathering, vulnerability analysis, password auditing, and website security testing.

Through the practical use of **Zenmap**, **Cisco Global Exploiter**, **Johnny (John the Ripper)**, and **OWASP ZAP**, we learned how security tools can identify potential weaknesses and help improve system protection. The project also improved our understanding of penetration testing concepts, network security, and defensive security measures.

The results showed the importance of regular security testing, system monitoring, strong password policies, and proper network protection. In addition, working with Kali Linux provided valuable hands-on experience with real cybersecurity tools used by security professionals.

Overall, this project highlighted the importance of cybersecurity testing in protecting systems and reducing security risks. It also demonstrated how security analysis tools can support organizations in maintaining safer and more secure environments.